Project Progress Report

For

StudyGlobe - Study Abroad Preparation System

Course Title: Software Engineering Design Capstone Project

Course Code: SE 331

Submitted To

Rahat Uddin Azad

Lecturer

Dept. of SWE

Daffodil International University

Submitted By: Team-03 (Undecided)

Student Name	Student ID
Md. Fazly Rabby	0242310005341077
Md. Munjurul Islam	0242310005341104
Kawshik Kumar Saha	0242310005341289

Batch: 40

Section: E1

Department of Software Engineering

Daffodil International University

Submission Date: 03/04/2026

Project Progress Report

StudyGlobe — Study Abroad Preparation System

1. Implemented Features

The following table lists all implemented features along with their current status:

SL no	Feature	Description	Status
1	User Registration	Secure sign-up form with full name, email, password, confirm password. Server-side validation (at least 6 char password). Creates user in Supabase Auth, custom users table, and user_profiles table	Completed
2	User Login & Authentication	Email/password login via Supabase Auth. Returns JWT access_token. Session stored in localStorage with auto-persistence across page refreshes.	Completed
3	User Logout	Clears session from Supabase and localStorage. Redirects to login page.	Completed
4	User Profile Management	View and edit profile: date of birth, phone, current education level, field of interest, preferred countries (TEXT[ ] array), budget range, target intake. Auto-creates profile row on first access.	Completed
5	Dashboard	Personalized landing page with welcome message. Quick-access cards to Countries, Universities, Programs, and Profile sections.	Completed
6	Country Catalog	Browse all available study destination countries. Displays country name, code, and description. Click navigates to filtered university list.	Completed
7	View Country-wise Universities	University listing with filter by country ID. Displays Name, city, type, website, and country name. Navigates to university details.	Completed
8	University Details — Programs Tab	Under university details, shows all programs offered with degree, field, duration, and tuition.	Completed
9	Academic Program Exploration	List all programs with university name joined. Filter by degree level and field of study. Displays degree, field, duration, and tuition per year.	Completed
10	View Intake Schedule	Under university details, Intake tab displays intake_name and start_month in table format.	Completed
11	Language Requirement Inspection	Under university details, Language tab displays test_name and min_score for each requirement.	Completed

12	Scholarship Eligibility Assessment	Under university details, Scholarships tab displays eligibility_basis, minimum_gpa, and additional_notes.	Partially Completed
13	Admin Data Management	Admin-only write policies enforced through Row Level Security (RLS) at database level.	Partially Completed
14	Preference-Based Filtering	Universities filterable by country. Programs filterable by degree level and field of study.	Completed
15	Navigation Bar	Responsive top navbar with auth aware links: shows Dashboard, Countries, Universities, Programs, Profile when logged in; Login/Register when logged out.	Completed

2. Mapping to SRS Requirements

The following table maps each implemented feature to the corresponding SRS Functional Requirement, ensuring clear alignment between the SRS and implementation:

FR ID	SRS Requirement	Implementation	Files Involved
FR-01	Secure Enrollment & Authentication	Supabase Auth sign-up/sign-in with JWT tokens. Custom users table for role management. Session persistence via localStorage.	authController.js, auth.js (middleware), AuthContext.js, Login.js, Register.js, ProtectedRoute.js
FR-02	User Profile Management	GET/PUT endpoints for user_profiles with auto-row creation. Frontend form with 7 editable fields including TEXT[ ] array support for preferred countries.	profileController.js, profile.js (route), Profile.js
FR-03	Country Catalog	GET endpoints for countries with full CRUD. Frontend grid display with country name, code, description. Links to university listing filtered by country.	countryController.js, countries.js (route), Countries.js
FR-04	View Country-wise Universities	GET /universities with optional country_id. Relational join with countries table for country name. Filter panel on frontend.	universityController.js, universities.js (route), Universities.js
FR-05	University Details	GET /universities/:id with full relational data (programs, intakes, language_requirements, scholarship_eligibility).	universityController.js, UniversityDetails.js
FR-06	Academic Program Exploration	GET /programs with filters (degree_level, field). Programs tab in university details	programController.js, programs.js (route), Programs.js, UniversityDetails.js

FR-07	View Intake Schedule	Intakes table in database. Data fetched as part of university details JOIN. Displayed in Intake tab with table format (intake_name, start_month).	setup.sql (intakes table), universityController.js, UniversityDetails.js
FR-08	Language Requirement Inspection	language_requirements table in database. Fetched via university details JOIN. Language tab displays test_name and min_score.	setup.sql (language_requirements), universityController.js, UniversityDetails.js
FR-09	Scholarship Eligibility Assessment	scholarship_eligibility table in database. Fetched via university details JOIN. Scholarships tab displays eligibility_basis, minimum_gpa, additional_notes.	setup.sql (scholarship_eligibility), universityController.js, UniversityDetails.js
FR-10	Admin Data Management	Database-level RLS admin-only write policies. Backend middleware requireAdmin checks user_metadata. CRUD endpoints for countries, universities, programs.	setup.sql (RLS policies), auth.js (middleware), all controllers
FR-11	Preference-Based Filtering	University filter by country_id. Program filter by degree_level (ilike) and field (ilike). Filter panel UI on both Universities and Programs pages.	universityController.js, programController.js, Universities.js, Programs.js

3. Testing Plan

Test Cases (Table Format) including- functional test cases & basic validation cases:

Test ID	Test Steps	Test Data	Expected Output	Test Status
TC-01	Send POST to /api/auth/register with valid data	email: "test@gmail.com", password: "test123", full_name: "Test User"	201 Created — user created in Supabase Auth + users table + user_profiles	Pass
TC-02	Send POST to /api/auth/register with missing fields	email: "test@gmail.com" (no password, no name)	400 Bad Request — "Email, password, and full name are required"	Pass
TC-03	Send POST to /api/auth/register with short password	password: "abc" (less than 6 chars)	400 Bad Request — "Password must be at least 6 characters"	Pass
TC-04	Send POST to /api/auth/login with valid credentials	email: "student@gmail.com", password: "student123"	200 OK — returns user object + session with access_token	Pass

TC-05	Send POST to /api/auth/login with wrong password	email: "student@gmail.com", password: "wrongpassword"	401 Unauthorized — "Invalid email or password"	Pass
TC-06	Send POST to /api/auth/logout	Valid session	200 OK — "Logged out successfully	Pass
TC-07	Send GET to /api/profile with valid Bearer token	Authorization: Bearer <valid_token>	200 OK — returns profile data (phone, field_of_interest, etc.)	Pass
TC-08	Send GET to /api/profile without token	No Authorization header	401 Unauthorized — "Unauthorized: No token provided"	Pass
TC-09	Send PUT to /api/profile with profile data	phone: "+880 1234 567890", field_of_interest: "Computer Science", preferred_countries: "Canada, UK"	200 OK — "Profile updated successfully", preferred_countries auto-converted to TEXT[ ]	Pass
TC-10	Send GET to /api/universities (list all)	No parameters	200 OK — array of universities with country name joined	Pass
TC-11	Send GET to /api/universities/:id with valid name	Name: cand	401 Unauthorized — "Invalid country"	Pass
TC-12	Send GET to /api/universities?country_id=1	country_id: 1	200 OK — filtered list of universities for that country	Pass
TC-13	Send GET to /api/programs (list all)	No parameters	200 OK — array of programs with university Name joined	Pass
TC-14	Send GET to /api/programs?degree_level=Bachelor	degree_level: "Bachelor”	200 OK — filtered list of Bachelor-level programs	Pass
TC-15	Send GET to /api/programs/university/:universityId	universityId: 1	200 OK — programs for that specific university	Pass
TC-16	Send GET to /api/countries (list all)	No parameters	200 OK — array of all countries ordered by name	Pass
TC-17	Send GET to /api/countries/:id with valid ID	id: 1	200 OK — single country object	Pass

TC-18	Open Login page → enter valid credentials → click Sign In	Valid email and password	Redirects to Dashboard with welcome message	Pass
TC-19	Open Login page → enter invalid password → click Sign In	Valid email, wrong password	Displays error message: "Login failed"	Pass
TC-20	Open Register page → fill all fields → click Create Account	Full name, valid email, password (at least 6 chars), matching confirm password	Shows success message, redirects to login after 3 seconds	Pass
TC-21	Access /dashboard without authentication	No login session	Automatically redirected to /login with return location saved	Pass
TC-22	Load Universities page → click a university card	Valid university data in database	Navigates to university details with 5 tabs (Overview, Programs, Intake, Language, Scholarships)	Pass
TC-23	Load Profile page → fill/edit fields → click Save Profile	Authenticated user, various profile fields	Shows "Profile updated successfully" message	Pass
TC-24	Load Programs page → select "Bachelor" degree level → click Search	degree_level: "Bachelor"	Displays filtered program cards showing only Bachelor programs	Pass
TC-25	Click Logout button in navbar	Active session	Session cleared, redirected to /login, navbar shows Login/Register	Pass

- Functional test cases — Covering all API endpoints (authentication, profile, universities, programs, countries) and all frontend user flows (login, registration, navigation, filtering, profile management).
- Basic validation cases — Input validation for registration (missing fields, short password), authentication checks (no token, invalid token), and data filtering (empty results, valid/invalid IDs).

Test Summary: All 25 test cases passed. 17 backend API tests and 8 frontend UI tests were executed covering authentication, profile management, data retrieval, filtering, and route protection.

4. Individual Contribution

Each team member's contribution is detailed below, including modules worked on, files contributed, key challenges faced, and lessons learned during development.

Member-1. Kawshik Kumar Saha (Team Lead)

Role: Database Design, API Architecture, Documentation, Testing & Validation

Modules Worked On:

- Complete PostgreSQL database schema design (8 tables with proper relationships and constraints)
- Row Level Security (RLS) policy design — public read, user-own-profile, admin-only write
- REST API route architecture and endpoint design for all 5 modules (auth, profile, universities, programs, countries)
- Supabase client configuration (service_role key for bypassing RLS on backend)
- Comprehensive test plan creation covering all API endpoints and frontend flows
- API endpoint testing and validation (11 backend tests documented)
- Project documentation (README, setup guide, architecture overview)

Files Contributed:

- backend/database/setup.sql — Full schema: users, user_profiles, countries, universities, programs, intakes, language_requirements, scholarship_eligibility + all RLS policies
- backend/config/supabase.js — Supabase client initialization
- backend/routes/auth.js, profile.js, universities.js, programs.js, countries.js — All route definitions
- testing/test_plan.md — Complete test plan document
- testing/api_test_results.md — API test results with full request/response documentation
- README.md — Project overview, setup instructions, bug fix log, FR coverage table

Key Challenges Faced:

- Designing RLS policies that allow public read access on lookup tables while restricting writes to admin users only — required careful policy layering to avoid conflicts
- Handling the foreign key dependency chain (Supabase Auth → custom users → user_profiles) during registration — had to ensure proper insertion order
- Testing authenticated endpoints required extracting and managing JWT tokens from Supabase Auth responses

What I Learned During Development:

- Gained deep understanding of PostgreSQL Row Level Security and how RLS policies interact with Supabase's service_role vs anon keys
- Learned how to design a database schema that bridges Supabase Auth's built-in user management with custom application tables
- Learned the distinction between Supabase anon key (RLS-restricted) and service_role key (bypasses RLS) and when to use each
- Improved skills in API testing methodology and documenting test results for team communication

Member-2. Md. Fazly Rabby

Role: Frontend Development & UI/UX Design

Modules Worked On:

- Complete React.js frontend with 8 pages and 2 reusable components
- Authentication flow — Login, Register forms with validation and error handling
- Dashboard with personalized welcome message and quick-access navigation cards
- University listing and details with 5-tab UI (Overview, Programs, Intake, Language, Scholarships)
- Program listing with degree level and field of study filters
- Country catalog with grid layout linking to filtered universities
- User profile management form with 7 editable fields
- Global auth state management using React Context API

Files Contributed:

- frontend/src/pages/Login.js — Login form with error handling, redirect logic
- frontend/src/pages/Register.js — Registration form with validation
- frontend/src/pages/Dashboard.js — Main dashboard with navigation cards
- frontend/src/pages/Profile.js — Profile view/edit form
- frontend/src/pages/Universities.js — University listing with filters
- frontend/src/pages/UniversityDetails.js — 5-tab university detail view
- frontend/src/pages/Programs.js — Program listing with filters
- frontend/src/pages/Countries.js — Country catalog
- frontend/src/components/Navbar.js — Auth-aware navigation bar
- frontend/src/components/ProtectedRoute.js — Route guard component
- frontend/src/context/AuthContext.js — Global auth state
- frontend/src/services/api.js — Axios client with interceptors

- frontend/src/App.js — Route definitions and layout

Key Challenges Faced:

- React Router v6 migration — <Redirect> component was removed, had to use <Navigate> instead
- Session persistence across page refreshes — sessions were lost because there was no localStorage integration; had to implement manual persistence in AuthContext
- The API calls were returning 401 because the Bearer token was never being attached — had to implement the Axios request interceptor
- Handling the preferred_countries TEXT[ ] array conversion between frontend string and PostgreSQL array format

What I Learned During Development:

- Deep understanding of React Context API for global state management and how it replaces Redux for simpler use cases
- Learned how to implement Axios interceptors for automatic token attachment and error-based redirects
- Improved skills in building responsive UIs with Tailwind CSS and creating tabbed interfaces
- Understood the importance of proper session management in single-page applications

Member-3. Md. Munjurul Islam

Role: Backend Development & Data Integration

Modules Worked On:

- Express.js server setup with JSON parsing, route mounting, and error handling
- Authentication controller — register (3-step: Supabase Auth + users table + user_profiles), login (JWT session), logout
- Profile controller with ensureUserRow() pattern for FK dependency resolution and TEXT[ ] array handling
- University controller with full relational JOINs (countries, programs, intakes, language_requirements, scholarship_eligibility)
- Program controller with university join and ilike filtering
- Country controller with full CRUD operations
- JWT authentication middleware using supabase.auth.getUser() token validation
- Admin authorization middleware with user_metadata role checking

Files Contributed:

- backend/server.js — Server entry point with CORS, JSON, routes, 404 and error handlers
- backend/controllers/authController.js — Auth logic (register, login, logout)
- backend/controllers/profileController.js — Profile CRUD with ensureUserRow
- backend/controllers/universityController.js — University CRUD with relational JOINs
- backend/controllers/programController.js — Program CRUD with filters
- backend/controllers/countryController.js — Country CRUD
- backend/middleware/auth.js — JWT validation and admin authorization
- backend/routes/ — All 5 route definition files
- backend/package.json — Dependency management

Key Challenges Faced:

- The original auth middleware used jwt.verify() with a static secret, which doesn't work with Supabase Auth tokens — had to switch to supabase.auth.getUser(token)
- Profile updates were failing because the FK relationship between user_profiles and users required an existing users row — solved with the ensureUserRow() upsert pattern
- The Supabase config originally used the anon key, which blocked backend operations due to RLS — had to switch to service_role key
- Handling edge cases where Supabase Auth user exists but custom users table row doesn't, requiring graceful fallback logic.

What I Learned During Development:

- Gained experience in building RESTful APIs with Express.js and Supabase JS SDK
- Understood the importance of middleware ordering (CORS → JSON → Routes → Error handlers) in Express
- Improved debugging skills when working with relational data and foreign key dependencies across multiple tables.